

Project: Fetch 3 APIs in Parallel + Retry Mechanism

Project Scenario

Tum ek news aggregator app banana rahe ho.

App ko simultaneously 3 different APIs se data fetch karna hai:

- JSONPlaceholder: users, posts, comments (free test API).

Parallel fetching (Promise.all), retry mechanism (exponential backoff),

timeout handling (Promise.race), partial failure handling (allSettled),

aur callback-based legacy code ko Promises mein convert karna — sab ek saath.

Part 1 — API Client with Promisification

Callback-based XMLHttpRequest ko Promise mein wrap karo (fetch ke bina!).

api-client.js — Promisified HTTP

```
// api-client.js
// Promisify XHR (old school – shows the concept clearly)
function xhrGet(url) {
  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', url);
    xhr.onload = () => {
      if (xhr.status >= 200 && xhr.status < 300) {
        try { resolve(JSON.parse(xhr.responseText)); }
        catch(e) { reject(new Error('JSON parse failed')); }
      } else {
        reject(new Error(`HTTP ${xhr.status}: ${xhr.statusText}`));
      }
    };
    xhr.onerror = () => reject(new Error('Network error'));
    xhr.send();
  });
}

// Modern: fetch already returns Promise – add timeout
const BASE = 'https://jsonplaceholder.typicode.com';

function fetchWithTimeout(url, ms = 5000) {
  const timeoutP = new Promise((_, reject) =>
    setTimeout(() => reject(new Error(`Timeout: ${url}`)), ms)
  );
  return Promise.race([fetch(url).then(r => {
```

```

if (!r.ok) throw new Error(`HTTP ${r.status}`);
return r.json();
}), timeoutP]);
}

// Export clean API
export const api = {
  getUsers: () => fetchWithTimeout(`${BASE}/users?_limit=5`),
  getPosts: () => fetchWithTimeout(`${BASE}/posts?_limit=10`),
  getComments: () => fetchWithTimeout(`${BASE}/comments?_limit=10`),
  getUserById: id => fetchWithTimeout(`${BASE}/users/${id}`),
};

```

Part 2 — Retry Mechanism

retry.js — Generic Retry Wrapper

```

// retry.js
const delay = ms => new Promise(res => setTimeout(res, ms));

export function withRetry(fn, { retries = 3, baseDelay = 500, factor = 2 } = {}) {
  return function(...args) {
    return new Promise((resolve, reject) => {
      function attempt(n) {
        fn(...args)
          .then(resolve)
          .catch(err => {
            if (n >= retries) {
              reject(new Error(
                `Max retries (${retries}) exceeded. Last error: ${err.message}`
              ));
            }
            return;
          })
        const wait = baseDelay * Math.pow(factor, n);
        console.warn(`Attempt ${n+1} failed. Retrying in ${wait}ms...`);
        delay(wait).then(() => attempt(n + 1));
      };
      attempt(0);
    });
  };
}

// Usage: wrap any async function
import { api } from './api-client.js';
export const reliableApi = {
  getUsers: withRetry(api.getUsers, { retries: 3, baseDelay: 300 }),
  getPosts: withRetry(api.getPosts, { retries: 3, baseDelay: 300 }),
  getComments: withRetry(api.getComments, { retries: 3, baseDelay: 300 }),
};

```

Part 3 — Parallel Fetch + Partial Failure (allSettled)

data-loader.js — Parallel with Partial Failure

```
// data-loader.js
import { reliableApi } from './retry.js';
export function loadDashboardData() {
  console.log('Loading dashboard data...');
  const start = Date.now();
  return Promise.allSettled([
    reliableApi.getUsers(),
    reliableApi.getPosts(),
    reliableApi.getComments(),
  ]).then(([usersR, postsR, commentsR]) => {
    const elapsed = Date.now() - start;
    console.log(`All fetches completed in ${elapsed}ms`);
    const result = { errors: [] };
    if (usersR.status === 'fulfilled') {
      result.users = usersR.value;
      console.log(`Loaded ${usersR.value.length} users`);
    } else {
      result.users = [];
      result.errors.push(`Users: ${usersR.reason.message}`);
      console.warn('Users failed:', usersR.reason.message);
    }
    if (postsR.status === 'fulfilled') {
      result.posts = postsR.value;
      console.log(`Loaded ${postsR.value.length} posts`);
    } else {
      result.posts = [];
      result.errors.push(`Posts: ${postsR.reason.message}`);
    }
    if (commentsR.status === 'fulfilled') {
      result.comments = commentsR.value;
    } else {
      result.comments = [];
      result.errors.push(`Comments: ${commentsR.reason.message}`);
    }
    return result;
  });
}
```

Part 4 — Main App: Chain It All Together

app.js — Full Integration

```
// app.js
import { loadDashboardData } from './data-loader.js';
function renderDashboard(data) {
  if (data.errors.length > 0) {
    console.warn('Partial load – some APIs failed:', data.errors);
  }
}
```


